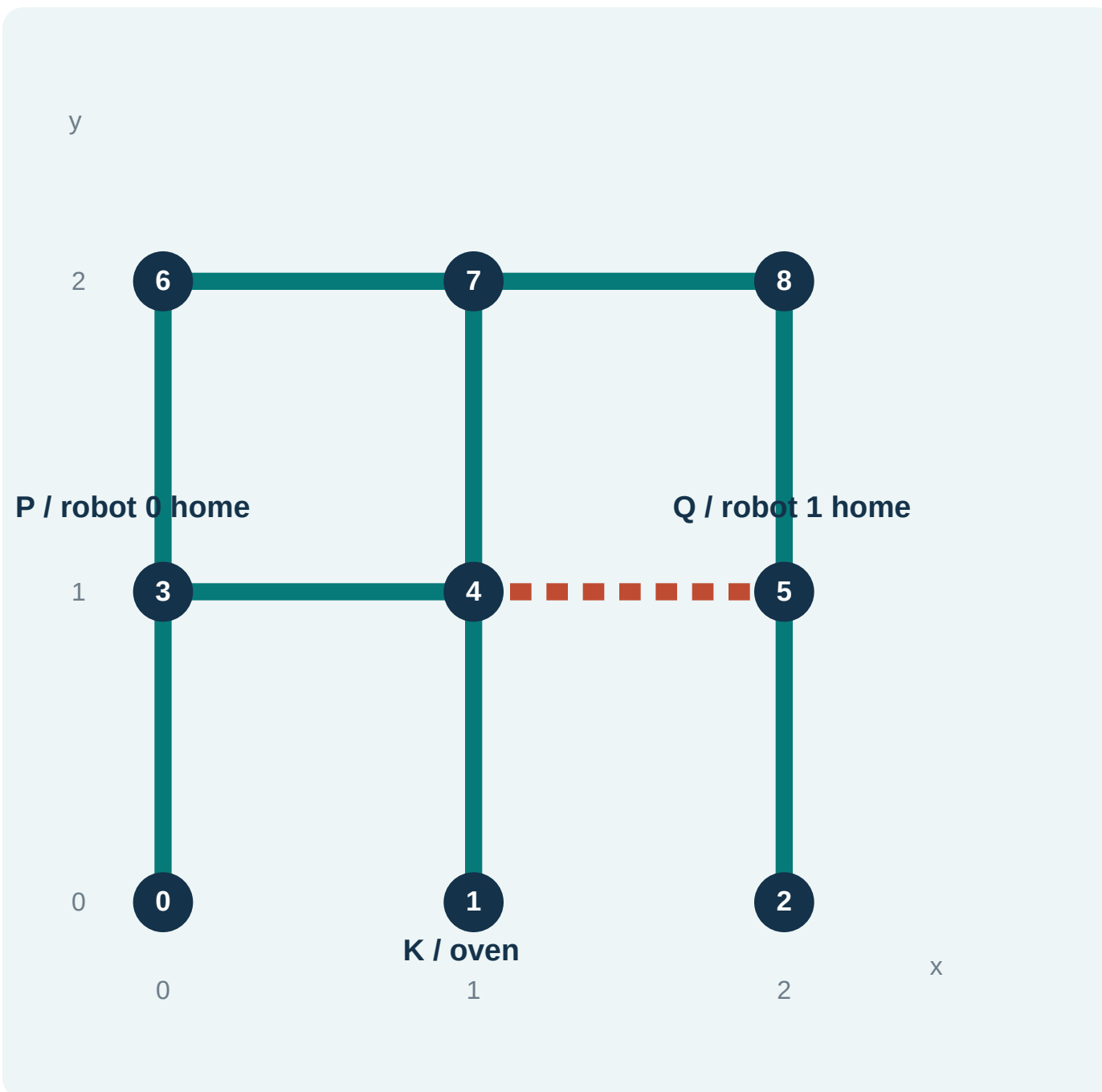


KILNWORKS / production planning

A / shared aisle geometry B / time and initial state



A. Read the graph, then route jointly

Node ID = $3y+x$. Each drawn segment is one edge and takes one minute. Moves follow one edge; the grid gives coordinates, not permission to use an undrawn edge. All dots are junctions. No diagonals.

P, Q and K are docking bays of capacity two robots. Every other node has capacity one. At every integer boundary robot counts obey these capacities.

In a minute at most one robot traverses any undirected edge: both swaps and same-direction sharing are forbidden. Following a robot into a node it leaves is legal. Pickup, unloading and waiting stay at the node.

Both robots are identical except for their fixed labels. Robot 0 starts at P (node 3); robot 1 at Q (node 5). Both start empty. Empty robots may reposition freely. Routes are decision variables; distance-only trips do not represent this plant.

B. Integer time

At boundary t , finish all operations ending at t , then choose all new starts and robot actions simultaneously. These occupy $[t, t+1)$; advance to $t+1$. Duration- d operations occupy $[t, t+d)$. Newly released resources are usable at that boundary.

Solid: always present. Dashed: present only for open-aisle designs. Every drawn segment joins its endpoint nodes in both directions. The map itself defines connectivity.

B. Start fresh; preserve state while idle

Each design/campaign is independent. All lots initially unstated; machines and oven empty; machine family memories unset; fixtures free. Waiting changes no state. No failures, resets, preemption or discards occur. Intentional idle time is legal. Stop at the boundary when every lot has finished curing.

C. Generate every lot

Campaign s is one of 0,1,2,3. Each has five lots $j=0,1,2,3,4$. All IDs and time indices are zero-based. For each lot calculate:

family $f = (j+s) \bmod 2$

release $r = 2 \text{ floor}(j/2)$ minutes

P processing = $5 + ((j*j+2s) \bmod 4)$ minutes

Q processing = $4 + ((3j+s) \bmod 5)$ minutes

Choose exactly one preparation machine for each lot. A start needs its release time reached, that machine idle, and one free fixture. Initial setup is zero. Later setup is two minutes for a family change on that machine, otherwise zero. Update its family memory at start.

Setup plus processing is one uninterrupted operation. P draws two power units; Q draws three throughout setup and processing. At completion the machine becomes free and the lot waits at that docking bay. Waiting output does not block preparation; buffers are unlimited.

D. Fixtures connect all three stages

A fixture is seized at preparation start and held through preparation, pickup waiting, transport, oven waiting and curing. Only cure completion frees it. F identical fixtures exist. A carried or waiting lot still owns its fixture.

D. One action per robot per minute

Wait: stay at the node; power zero.

Move: traverse one drawn edge to its other endpoint, with or without cargo; power one.

Pickup: an empty robot at P or Q takes one prepared, untransported lot at that machine. It stays at the dock and draws one power unit. The lot becomes cargo at $t+1$.

Unload: a loaded robot at K releases its lot into oven waiting. It stays at K and draws one power unit. The lot becomes available to cure at $t+1$.

Each robot carries at most one lot. Two robots cannot pick up the same lot. A prepared lot may be picked up at its completion boundary, but a pickup started at t does not move in that minute. An unload ending at t may join a batch starting at t . Cargo can wait on a robot. Locations persist; robots never teleport.

E. One oven, fixed batches

At an oven start select one or two arrived, uncured lots of the same family. The oven must be idle. Cure time is 4+family minutes; power is three throughout. Batch membership is fixed at start. Every lot in a batch completes together and frees its fixture. Arrivals do not start curing automatically.

F. Aggregate power in each minute

Sum power of active preparation operations, active curing, and both robot actions. It must be at most G. Check the whole simultaneous start/action set, including operations already running. Setup draws full machine power. Idle machines and waiting lots draw zero.

F. Two exact schedule objectives

Makespan M is the final cure-completion boundary. Tariff during [t,t+1) is $1+(\text{floor}(t/7) \bmod 3)$. Bill = sum over t=0..M-1 of tariff times total active power. Time is minutes; power and currency are exact benchmark units.

For each design/campaign minimize (M,bill) lexicographically: first the smallest feasible M; then the smallest bill among schedules at that M. Search all feasible machine choices, starts, batches and robot routes. Idle time is legal. Any schedule attaining both minima is equivalent; no action-order or path tie-break is imposed.

A feasible schedule is an upper bound. Establish exact optimality using a complete search or mathematically checkable certificate. A distance relaxation is a lower bound only; it cannot certify collision feasibility. All arithmetic is integer; no rounding is needed.

G. Six independent design IDs

ID	F	G	aisle	capital
D0	2	5	closed	0
D1	3	5	closed	7
D2	2	6	closed	9
D3	2	5	open	6
D4	3	6	closed	16
D5	3	6	open	22

Closed/open changes only the dashed edge in A. Baseline is D0. Capital cost is paid once per design; schedules still start fresh for each campaign.

Execute the complete six-design by four-campaign product. For each design form the selection key:

**(maximum of its four optimum makespans,
sum of its four optimum bills + 3 capital,
capital, design ID)**

Recommend the minimum key. Also select the minimum key restricted to capital <=9, including equality. Design ID breaks any remaining tie in ascending order. These are two populations, not contradictory recommendations.

Six logical deliverables

Optimizer source: executable offline code generating every campaign, searching all designs and writing the outputs. A documented local toolchain is sufficient; no specific language is required.

Independent verifier source: separately runnable code reconstructing constraint and objective checks. It shares immutable instance data only with the optimizer, not transition, feasibility or pruning routines.

Schedules: machine preparation starts, oven starts with lot membership, and every minute of both robot actions for all cases. Identify design, campaign, minute and lot IDs. State the node convention and robot identities.

Case matrix: one record per design/campaign with optimum makespan and bill. Include both selection results and their selection keys in this file or a companion section.

Certification evidence: complete optimality proof for each case, plus verifier output. This may be an independently executed exhaustive algorithm or a complete checkable certificate. State merging and pruning must be justified; a hash alone is not proof.

Engineering memo: graph interpretation, commands/versions, verification outcomes and the causal engineering recommendation described at right.

Production evidence and correction

Generate a chronological production trace from each delivered schedule. It must expose starts/completions, both robot positions and cargo, active machine/oven work, fixture count, per-minute power and cumulative bill. Group simultaneous actions; their print order is immaterial.

All traces, case metrics, proofs and recommendations must refer to those same uninterrupted schedules. Equivalent file nesting, column names, extra source files and compression are acceptable if the documented commands reconstruct the evidence.

Run the delivered verifier on all cases. It must independently check aisle moves, edge/node occupancy, cargo/lot conservation, release/precedence, setup, fixtures, batch compatibility, aggregate power and objective arithmetic, and independently establish every claimed optimum.

As an adversarial check, copy a schedule and shift its earliest preparation start to -1. The verifier must reject it. Also execute a route-collision mutation and report its first invalid minute. Preserve the unmodified production schedule.

In the memo compare D0 to both selected designs using executed objectives. Explain at least one actual routing/batching/resource interaction with event times from delivered schedules. Explain why a distance-only relaxation may understate achievable completion time. Describe one executed debugging or adversarial-check iteration and its result.